Reward Engineering Playbook: A Systematic Study of Dense Reward Geometry for Robot Manipulation

Anonymous Author(s)
Affiliation
Address
email

Abstract: Dense reward design for robot manipulation is a persistent bottleneck: small changes in reward formulas can produce qualitatively different learning outcomes, yet the field lacks principled guidance for making these choices. We present a systematic empirical study of dense reward geometry using ManiSkill3's tabletop manipulation suite as a controlled testbed, injecting reward variants as environment parameters and holding algorithms and tasks fixed to isolate causal effects. Our experiments establish two core findings. First, a fundamental *robustness-ceiling tradeoff*: infinite-support rewards (e.g. `tanh`) trap policies in *hover attractors*—stable sub-optimal states where the shaped reward is high but task completion is not—while finite-support rewards (`concave_truncated`) resist this but create blind regions for hard initialisations. Within-episode annealing resolves the tradeoff on flat tasks (PickCube: 0.979, PushCube: 0.958, LiftPegUpright: 0.854) but catastrophically fails on hierarchical tasks (StackCube: mean peak 0.063 vs. 1.000 for finite-support)—a complete ranking inversion caused by the time clock decoupling from task progress at stage boundaries. Second, this failure motivates a *theoretically grounded* alternative: `adaptive_concave_distance`, a closed-form static reward in which the support radius self-adapts to the agent's current distance from the goal via $k(d) = k_{\min} + (k_{\max} - k_{\min})e^{-\alpha d}$. We derive conditions on $k_{\min}$, $k_{\max}$, α governing the reward's geometry, and show empirically that it matches `concave_truncated`'s peak performance on StackCube (mean peak 0.979 vs. 1.000) while converging approximately $2\times$ faster. The result is a *reward engineering playbook* with a theoretically principled base function for both flat and hierarchical tasks.

Keywords: reward shaping, dense rewards, robot manipulation, reinforcement learning, reward design, empirical study

1 Introduction

Reinforcement learning for robot manipulation requires dense reward functions to guide exploration: purely sparse rewards that fire only at success are rarely sufficient for contact-rich, multi-stage tasks. Yet practitioners designing these dense rewards have few principled guidelines. The standard approach is to write a distance-based potential that decreases toward the goal, add binary stage bonuses, and tune scale factors empirically until training converges. This process is slow, non-transferable, and yields little understanding of which mathematical properties of the reward function matter.

The question we study is: *given that the task and algorithm are fixed, how does the geometry of the dense reward function—its shape, curvature, support, smoothness, and staging—causally affect learning dynamics and final performance?*

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

This is distinct from several related lines of work. Automated reward generation methods [1] improve the speed of reward iteration but treat reward code as a black box to be searched rather than a structured object to be understood. Reward shaping theory [2, 3] provides guarantees about policy invariance under specific transformations but says little about optimization speed or empirical ranking among non-equivalent reward functions. What is missing is a controlled, systematic empirical study of reward geometry under on-policy learning in manipulation.

We address this gap by decomposing dense manipulation rewards into four independently parameterizable dimensions (§3), implementing them as environment kwargs in ManiSkill3 (§4), and sweeping across combinations while holding everything else fixed.

Contributions.

- A taxonomy of dense reward geometry along four independently controllable axes, implemented as environment kwargs across 11 ManiSkill3 tabletop tasks (Appendix A).
- Identification and characterisation of the *hover attractor*—a failure mode of infinite-support rewards—and the fundamental *robustness-ceiling tradeoff* between hover attractors and blind regions.
- A mechanistic account of why within-episode time annealing succeeds on flat tasks but catastrophically fails on hierarchical ones (complete ranking inversion on StackCube).
- A first-principles theory of distance-adaptive reward geometry (Section 5), deriving closed-form conditions on the reward's support structure, moat existence, and parameter roles.
- `adaptive_concave_distance`: a principled static reward derived from the theory that achieves comparable peak performance to the best task-specific static reward while converging $\approx 2\times$ faster on StackCube-v1 (mean peak 0.979).
- A reward engineering playbook: empirically grounded, theoretically motivated guidance for dense reward selection by task structure.

2 Related Work

Reward shaping theory. Ng et al. [2] established that potential-based shaping does not alter the optimal policy in the limit, but this result does not address optimization dynamics under finite-sample on-policy methods. Devlin and Kudenko [3] extended the theory to dynamic potential functions. Our work is empirical and focuses on how shaping geometry affects convergence speed and stability rather than asymptotic correctness.

Automated reward design. EUREKA [1] uses LLMs to generate reward code and refine it via evolutionary search, achieving impressive results across locomotion and manipulation tasks. However, the method treats reward formulas as programs to be searched, not as geometric objects to be understood. Our work is complementary: we aim to understand *why* certain reward structures work, which could inform better search spaces for automated methods.

Empirical studies of RL design choices. Andrychowicz et al. [4] conducted a large-scale study of on-policy RL design choices (architecture, normalization, etc.) but did not focus on reward geometry. To our knowledge, no comparable study exists for manipulation-specific reward function design.

Simulation infrastructure. ManiSkill3 [5] provides GPU-parallelized simulation, a standardized API across 18+ tabletop manipulation tasks, and built-in support for PPO baselines, making it ideal for controlled reward geometry sweeps.

Hierarchical task reward design. DrS [**?**] learns reusable dense reward functions for multi-stage tasks by distilling sparse success signals into per-stage dense rewards. Like our work, it studies stage-decomposed reward design; unlike ours, it focuses on reward *magnitude* learned from data, not

the *functional form* (curvature, support) analyzed analytically. Hierarchical Potential-Based Reward Shaping (HPRS) [?] extends Ng et al.'s potential-based framework to task specifications, but again studies magnitude via potential functions rather than geometry. Self-Adaptive Reward Shaping [?] adapts reward weights online but does not study the geometric properties of the shaping function itself.

Hierarchical reinforcement learning. Option-Critic [?] and HIRO [?] learn temporal abstractions (options and sub-goals respectively) to decompose hierarchical tasks. Our approach is complementary: task stages are *given* by the task's `evaluate()` function, so we do not need to learn when to switch stages. We study how the reward geometry within each known stage affects learning. HER [?] uses goal-conditioned relabeling to handle sparse rewards; our work instead investigates how the dense shaping geometry interacts with stage structure.

Curriculum learning. Narvekar et al. [?] provide a framework for curriculum RL in which training instances are ordered by difficulty. Our within-episode annealing (`adaptive_tanh_episode`) is a special case of curriculum learning where the geometry of the reward function itself changes within an episode. The stage-conditioned extension (`adaptive_tanh_stage`) conditions the curriculum on task progress rather than wall-clock time. From Sparse to Dense [?] motivates sparse-to-dense reward transitions from biological learning; our adaptive annealing formalizes this intuition as a within-episode geometric schedule.

Concurrent work. Several 2026 preprints explore related but distinct directions: reward landscape analysis for model-based planning, adaptive reward weighting, and two-stage curriculum design. These address different algorithmic levers (planning objectives, weight scheduling, task ordering) rather than the intrinsic geometry of the dense shaping function itself.

3 Reward Geometry Taxonomy

We decompose dense manipulation rewards into four independently controllable dimensions. Each dimension is exposed as an environment kwarg (e.g., `reach_variant="concave_truncated"`), enabling factorial sweeps without modifying the algorithm or task.

3.1 Dimension 1: Reaching Shape

The reaching component $r_d : \mathbb{R}_{\geq 0} \to \mathbb{R}$ maps end-effector-to-object distance d to a scalar reward. We study eight static families and three adaptive variants that vary in curvature, support, and monotonicity (Table 1).

Table 1: Reaching reward shape variants. All satisfy $r_d(0) = 1$. $\nabla r_d|_{d=0}$ is the gradient at the goal.

Name	**Formula**	**Curvature**	$\nabla r\|_{d=0}$
`tanh`	$1 - \tanh(5d)$	Convex	-5
`linear`	$\max(0, 1 - 5d)$	Linear	-5
`exponential`	e^{-5d}	Convex	-5
`quadratic`	$\frac{1}{1+25d^2}$	Concave peak	0
`tanh_sq`	$1 - \tanh(25d^2)$	Concave, no cutoff	0
`concave_truncated`	$\max(0,\ 1 - 25d^2)$	Concave, $r = 0$ for $d > 0.2$	0
`oscillatory`	$(1 - \tanh(5d)) + 0.05\sin(20d)$	Non-monotone	-5
`sparse_threshold`	$\mathbf{1}[d < 0.05]$	Discontinuous	—
`adaptive_tanh_episode`	$1 - \tanh(k(t) \cdot d)$, $k(t) = 2{+}18\frac{t}{T}$	Convex, time-varying	$-k(t)$
`adaptive_tanh_stage`	$1 - \tanh(k(t_s) \cdot d)$, k resets per stage	Convex, stage-varying	$-k(t_s)$
`adaptive_concave_distance`	$\max\big(0,\ 1 - (k(d){\cdot}d)^2\big)$, $k(d) = k_{\min} + \Delta k\, e^{-\alpha d}$	Concave, k varies with d	0

3.2 Dimension 2: Stage Gating

Many manipulation tasks are hierarchical: the robot must reach, then grasp, then place. We study how the placement reward r_p is conditioned on the grasping phase variable $c_t \in \{0, 1\}$ (or a continuous proxy):

- `hard`: $r_p \cdot c_t$ (binary mask; baseline)
- `soft`: $r_p \cdot \sigma(k(\frac{1}{2} - \text{openness}_t))$ (continuous gripper closure)
- `additive`: r_p (no gate; all stages always active)
- `curriculum`: hard gate for first 40% of episode, then additive

3.3 Dimension 3: Terminal Bonus

At task success, we study three treatments of the terminal reward:

- `hard_jump`: override $r_t = r_{\max}$ at success (baseline)
- `smooth`: additive bonus $r_t \mathrel{+}= 0.5\, r_{\max} \cdot \mathbf{1}[\text{success}]$
- `none`: pure shaping; no terminal override

3.4 Dimension 4: Normalization

[TODO: Planned: unnormalized dense, normalized $[0, 1]$, z-score per batch, clipped. Currently using ManiSkill3's built-in normalized dense reward (divide by $r_{\max}$).]

4 Experimental Setup

Tasks. We study four tasks spanning the main structural archetypes in tabletop manipulation: PickCube-v1 (canonical pick-and-place, flat), PushCube-v1 (pushing without grasping, flat), LiftPegUpright-v1 (rotation-dominated, flat), and StackCube-v1 (two-stage hierarchical). PlaceSphere-v1 (three-stage hierarchical) is planned next. A full listing of the 11 tasks with reward variant support is in Appendix A.

Algorithm. All experiments use PPO [6] via the ManiSkill3 fast GPU baseline (256 parallel environments, 50-step rollouts). Flat tasks use 5M total timesteps; StackCube-v1 uses 40M to allow the harder hierarchical task sufficient training budget. We plan to extend to SAC [7] and a model-based method to assess algorithm-dependence of our findings.

Infrastructure. Reward variants are injected as environment kwargs, requiring no algorithm modification. This is implemented in a dedicated `reward_variants` module with three independent axes.

Evaluation. Each variant is evaluated with a *deterministic* policy (mean action, no noise) on 16 parallel environments every 25 training iterations. Primary metric: `eval/success_once`. Secondary metrics: eval return, explained variance, gradient norm, value loss, and steps to 80% success. Train/eval success gap is tracked as a reward hacking diagnostic.

Compute. RTX 4070 laptop (8GB VRAM). At 256 envs and 5M steps, one run takes approximately 45–60 minutes. Multi-seed validation (3 seeds per configuration) is feasible within the project timeline.

5 Theory of Reward Geometry

5.1 From Static to Adaptive k: The Design Progression

The experiments in Section 6 trace a natural arc in the design of reaching rewards. The concave truncated baseline has the form

$$r_d(d) = \max\big(0,\, 1 - (k\,d)^2\big), \tag{1}$$

with fixed k. This is a concave parabola that is exactly zero beyond $d = 1/k$: the *support radius* is $R = 1/k$. The core problem (F3) is that R must simultaneously be large enough to provide gradient signal from hard initialisations and small enough to prevent a soft hover attractor near the goal. No single value of k satisfies both.

The time-adaptive variants (`adaptive_tanh_episode`, `adaptive_tanh_stage`) address this by making k a function of elapsed time within an episode, starting small (wide R) and growing large (tight R). This resolves the tradeoff on flat tasks but fails on hierarchical ones because *time is a poor proxy for task progress*: by mid-episode on StackCube, the time clock has already tightened k while the robot is still in stage 1 (F10).

The natural resolution is to let k depend directly on the current distance d:

- **When far from the goal** (d large): use small k (wide R) to ensure the policy receives a gradient regardless of initialization.
- **When close to the goal** (d small): use large k (tight R) to create strong inward pull and suppress the soft hover attractor.

This gives a reward that is *geometrically self-adapting*: the support radius $R(d) = 1/k(d)$ automatically widens or narrows based on where the policy is, with no internal state or time clock.

5.2 The Distance-Adaptive Reward Function

We parametrize $k(d)$ with an exponential interpolation between two extreme values:

$$k(d) = k_{\min} + (k_{\max} - k_{\min})\, e^{-\alpha d}, \tag{2}$$

so $k(0) = k_{\max}$ (tight near goal) and $k(d) \to k_{\min}$ as $d \to \infty$ (wide far from goal). The full reward is

$$\boxed{r(d) = \max\Big(0,\; 1 - \big(k(d)\cdot d\big)^2\Big), \quad k(d) = k_{\min} + (k_{\max} - k_{\min})\, e^{-\alpha d}.} \tag{3}$$

We use $k_{\min} = 2$, $k_{\max} = 20$, $\alpha = 10$ in all experiments reported here; these are the first configuration we tried and further tuning may yield better results.

REPCon and REPTan units. We give a name to this primitive. A **REPCon unit** is an instance of Eq. (3) — a concave distance-adaptive reward characterised by the triple $(k_{\min}, k_{\max}, \alpha)$. The triple encodes the geometric requirements of a single subtask: $k_{\min}$ sets the outer support radius (how far the agent may start), $k_{\max}$ sets the inner basin tightness (the precision required at success), and α controls the transition length scale between the two regimes. Replacing the concave base $\max(0, 1 - (k(d)d)^2)$ with a tanh base $1 - \tanh(k(d) \cdot d)$ gives the analogous **REPTan unit**, which has infinite support (no hard cutoff) but otherwise the same $k(d)$ structure.

A hierarchical task with n stages can in principle be equipped with n REPCon (or REPTan) units, each tuned to the geometry of its subtask. The current `adaptive_concave_distance` variant is the degenerate case where all stages share a single REPCon unit — which already achieves strong results, suggesting a well-chosen unit generalises across subtasks. Per-stage parameter tuning is a natural extension (Section 7).

5.3 Geometry of the Reward: The Scaled Distance $\varphi(d) = k(d) \cdot d$

Because $r(d) = \max(0, 1 - \varphi(d)^2)$, the shape of the reward is entirely determined by the *scaled distance* $\varphi(d)$. Understanding r reduces to understanding φ.

Outer cutoff. The reward reaches zero when $\varphi(d) = 1$. For large d, $k(d) \approx k_{\min}$, so the outer cutoff occurs near $d = 1/k_{\min}$. With $k_{\min} = 2$, this is ≈ 0.5m: numerically, $r(d) = 0$ for $d \gtrsim 0.46$m. This is the effective *outer support radius*.

Inner basin. Near $d = 0$, $k(d) \approx k_{\max} = 18$, so the reward gradient is steep — equal to the gradient of $\max(0, 1 - (18d)^2)$, a tight parabola with effective radius $1/k_{\max} \approx 0.056$m. The policy thus experiences a strong inward pull within the last $\approx$ 6cm of approach.

Does a moat exist? $\varphi'(d) = k(d) + k'(d)\, d = k_{\min} + (k_{\max} - k_{\min})\, e^{-\alpha d}(1 - \alpha d)$. For $d > 1/\alpha$, the factor $(1 - \alpha d)$ is negative, so φ' can become negative (i.e., φ can have a local maximum). If φ has a local maximum φ^*, then:

- If $\varphi^* > 1$: the reward passes through zero twice, creating a genuinely *dead zone* (outer ring where $r = 0$) sandwiching an outer lobe of nonzero reward.
- If $\varphi^* < 1$: the reward never reaches zero at the local max — there is no dead zone, only a *shallow moat* (local minimum) in the reward profile, followed by a small outer bump.

The worst case of φ' occurs at $d^* = 2/\alpha$, where $\varphi'(d^*) = k_{\min} - (k_{\max} - k_{\min})e^{-2}$. Setting this to zero gives the moat-existence condition:

$$\frac{k_{\max}}{k_{\min}} > 1 + e^2 \approx 8.4. \tag{4}$$

With $k_{\min} = 2$, $k_{\max} = 20$: $k_{\max}/k_{\min} = 10 > 8.4$, so a moat *does* exist. Numerically, φ has a local max at $d \approx 0.165$m ($\varphi \approx 0.837$) and a local min at $d \approx 0.245$m ($\varphi \approx 0.828$), giving $r \approx 0.299$ at the moat and $r \approx 0.314$ at the outer bump. The moat is very shallow (depth ≈ 0.015) because $k_{\max}/k_{\min} = 10$ is only just above the threshold 8.4.

5.4 Parameter Roles

Each of the three parameters controls a distinct geometric property:

$k_{\min}$: outer support radius. $R_{\text{outer}} \approx 1/k_{\min}$. Decreasing $k_{\min}$ widens the region where the policy receives a nonzero gradient signal — improving robustness to hard initialisations. However, it also lowers the ratio $k_{\max}/k_{\min}$, which can eliminate the moat (Eq. (4)) and reduce the inner basin tightness. With $k_{\min} = 2$, $R_{\text{outer}} \approx 0.5$m.

$k_{\max}$: inner basin tightness and moat depth. $R_{\text{inner}} \approx 1/k_{\max}$ controls how tightly the gradient is concentrated near the goal. Increasing $k_{\max}$ sharpens the inner basin but also deepens the moat (increases φ^*). From Eq. (4) and $\Delta k = k_{\max} - k_{\min}$, the moat vanishes when $\Delta k \leq k_{\min}\, e^2$, equivalently $k_{\max} \leq k_{\min}(1 + e^2)$. For $k_{\min} = 2$ this threshold is $\Delta k \approx 14.8$ (i.e. $k_{\max} \approx 16.8$): with $\Delta k \leq 14$ the reward is a smooth, strictly decreasing function; with $\Delta k \geq 15$ a moat appears. Our configuration ($k_{\max} = 20$, $\Delta k = 18$) sits just above the threshold, producing a very shallow moat. With $k_{\max} = 20$, $R_{\text{inner}} \approx 0.05$m.

α: commitment distance. α controls how quickly $k(d)$ transitions from $k_{\max}$ (near goal) to $k_{\min}$ (far away), and in doing so sets the distance at which the policy transitions from the *exploration regime* (wide support, shallow gradient) to the *precision regime* (tight basin, steep gradient). Intuitively, α answers the question: *at what distance from the goal does the policy begin committing to precision rather than gross approach?* Small α means the transition is gradual and spread over a large distance — the reward tightens slowly throughout the entire approach. Large α means the transition is sharp

and localised: the policy experiences wide exploratory reward for most of the approach, then hits an abrupt shift into the precision basin close to the goal. The moat, when it exists, appears at $d \approx 1.65/\alpha$ (between $1/\alpha$ and $2/\alpha$), which is exactly the commitment boundary — the geometric point past which the policy is in the inner basin and should commit. With $\alpha = 10$, the commitment distance is $d \approx 0.165$m, well within the support radius (0.46m), and thus visible. Lowering α to 5 would push the commitment boundary toward $d \approx 0.35$m, where $\varphi > 1$ and the reward is already zero — the moat becomes invisible. Raising α to 15 moves the commitment boundary to $d \approx 0.11$m and deepens it to $r \approx 0.69$, making it a more substantial feature of the landscape.

We have not yet varied these parameters experimentally; the configuration $k_{\min} = 2$, $k_{\max} = 20$, $\alpha = 10$ is the first one we tested.

5.5 Why Distance-Adaptivity Resolves the Hierarchical Failure

The failure of time-based annealing on StackCube (F10) stems from the episode clock becoming decoupled from task progress at stage boundaries. Distance-based adaptivity eliminates this coupling entirely: $k(d)$ is a *pure function of the current robot–object distance*, with no memory of time, stages, or episode history.

When a stage transition occurs in StackCube (grasp achieved; now reach for placement target), d jumps from near-zero (successful grasp) to a large value (distance to placement location). This jump in d causes $k(d)$ to drop back toward $k_{\min}$, automatically widening the support radius for the new sub-goal — exactly the behavior that the stage-clock reset in `adaptive_tanh_stage` tried to engineer explicitly. Here it happens for free, as a consequence of the geometry.

More generally, `adaptive_concave_distance` is *geometrically adaptive but temporally static*: it is a closed-form function of d alone, with no internal state. The "adaptivity" lives in the reward landscape's curvature, not in any schedule. Table 2 places this in the context of the full design progression.

Table 2: Arc of reward design decisions and their failure modes.

Design	**Mechanism**	**Failure mode**
Static infinite (`tanh`)	Fixed wide R	Soft hover attractor
Static finite (`concave_truncated`)	Fixed narrow $R = 1/k$	Hard attractor / blind region
Time-adaptive (`adaptive_tanh_episode`)	$k(t)$: small→large over episode	$t \neq$ progress on hierarchical tasks
Stage-adaptive (`adaptive_tanh_stage`)	$k(t_s)$, reset per stage	Stage clock still decoupled from d
Distance-adaptive (`adaptive_concave_distance`)	$k(d)$: large→small with d	Requires parameter tuning

6 Results

All experiments use `gate=hard`, `terminal=hard_jump`, 256 parallel environments, seeds $\{1, 42, 123\}$.

6.1 PickCube-v1: Reach Shape Ablation

5M steps. Learning curves and 95% confidence intervals are shown in Figures 1–3. Table 3 reports final success rates.

F1 — Hover attractors cause initialisation-dependent failure. `tanh` and `tanh_sq` reach 1.0 on easy seeds (42, 123) but completely fail on seed 1 (Figure 2, left panel). Formally, let $d(s)$ be the distance to the goal and $r_d(d)$ be the dense reward. A state s_h with $d_h = d(s_h) > 0$ acts as a *hover attractor* if the expected gradient of the advantage function $A(s, a)$ pushes the policy toward maintaining d_h rather than progressing toward $d = 0$. For infinite-support rewards like $r_d(d) = 1 - \tanh(kd)$, the gradient $\nabla_d r_d(d) = -k\text{sech}^2(kd)$ exists and is non-zero everywhere. This creates a continuous gradient ascent path toward a local region where the shaping reward is

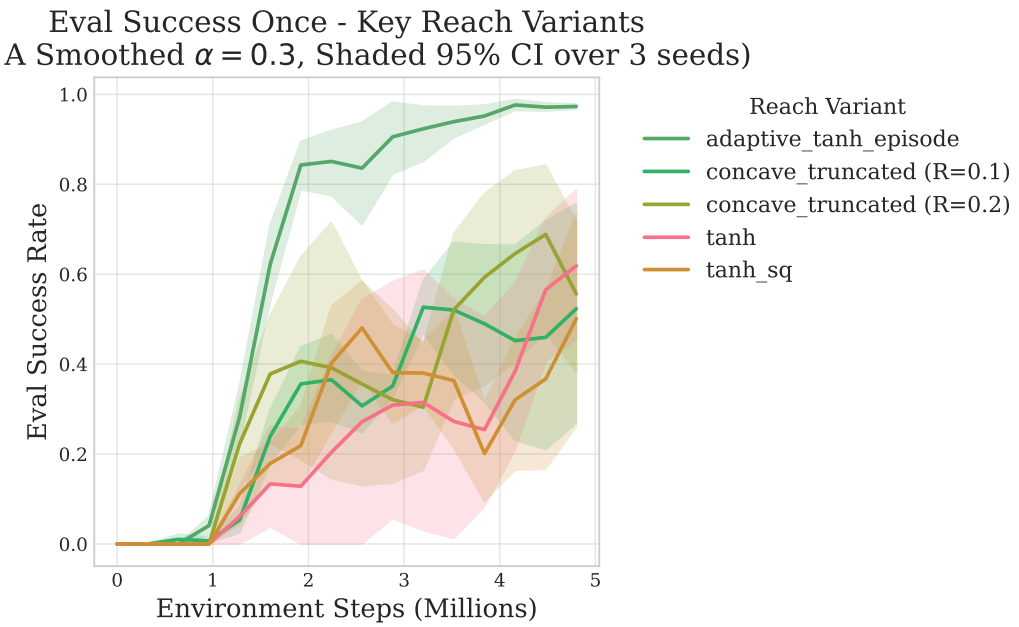

Figure 1: Key reach variants on PickCube-v1. Mean $\pm$ 95% CI over seeds $\{1, 42, 123\}$. `adaptive_tanh_episode` (green, bold) is the only variant that achieves near-perfect success consistently across all seeds.

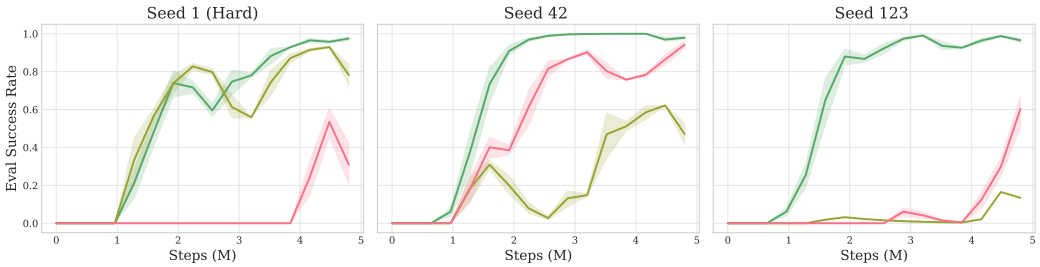

Figure 2: Per-seed learning curves for three representative variants. Seed 1 is a hard initialisation on which `tanh` and `concave_truncated` both struggle; `adaptive_tanh_episode` reaches 1.0 on all three seeds.

high, but crossing into the next task stage (e.g., grasping the object) requires a coordinated sequence of actions that temporarily lowers expected return. Thus, hard initialisations converge to this hover attractor ($d \approx 0.3$m) and never escape. Explained variance remains ≈ 0.96 throughout, confirming the failure is policy-level, not a value function issue.

F2 — Finite support prevents hover attractors but causes mode-switching instability. `concave_truncated` (R=0.2) survives seed 1 (peak 0.9375) but oscillates dramatically and ends at 0.625 after 5M steps. Mathematically, the abrupt zero at $R = 0.2$m in $r_d(d) = \max(0, 1 - (d/R)^2)$ creates a non-differentiable boundary in the state-reward mapping. When the policy π_θ is near this boundary, slight errors in the value function $V(s)$ cause the advantage estimate to swing wildly between positive and negative depending on whether a sampled trajectory crosses into the $d < R$ zone. This discontinuity in the incentive boundary forces the policy to repeatedly cross it in both directions, producing *mode-switching* (alternating between exploring outside the zone and exploiting inside it) rather than stable convergence. Narrowing to R=0.1 slows initial learning $\approx 2.5\times$ and produces the same instability.

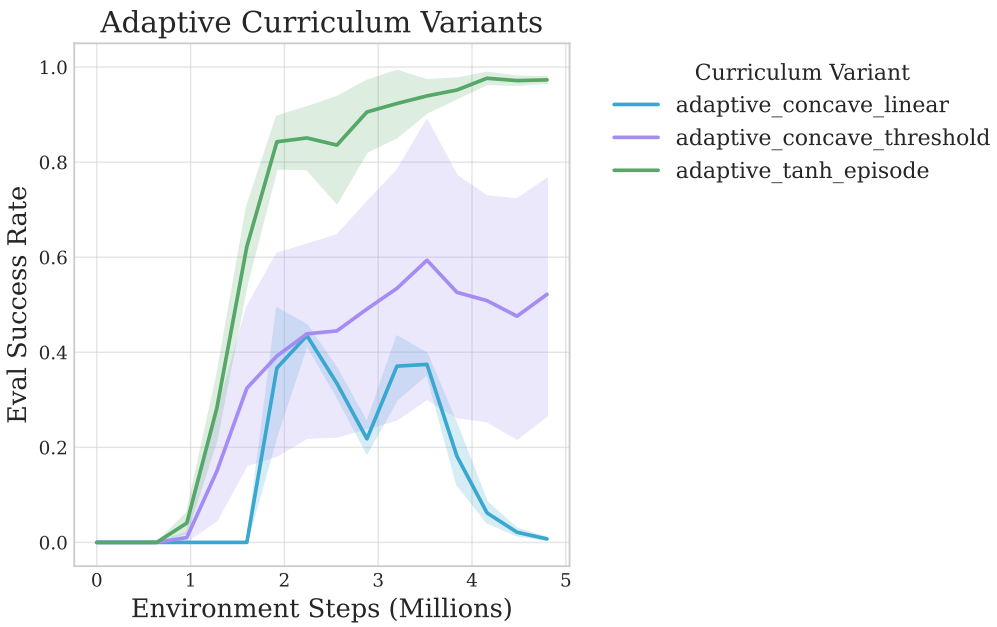

Figure 3: Adaptive curriculum variants versus static baselines. The linear schedule (`adaptive_concave_linear`) collapses because it shrinks the support radius continuously regardless of learning progress. The threshold schedule (`adaptive_concave_threshold`) fails on seed 1 due to a chicken-and-egg: it requires success to narrow the radius, but needs the narrow radius to achieve success. Within-episode annealing (`adaptive_tanh_episode`) breaks the deadlock by delivering the full wide-to-narrow curriculum in every episode from the start of training.

Table 3: Final `eval/success_once` at 5M steps (PickCube-v1). Seeds $\{1, 42, 123\}$ where available.

Variant	**Seed 1**	**Seed 42**	**Seed 123**	**Mean $\pm$ Std**
`tanh`	0.000	1.000	0.875	0.625 ± 0.445
`tanh_sq`	0.000	1.000	0.938	0.646 ± 0.457
`concave_truncated` (R=0.2)	0.625	0.312	—	0.469 ± 0.156
`concave_truncated` (R=0.1)	0.875	0.000	0.875	0.583 ± 0.412
`adaptive_concave_linear`	—	0.000	—	0.000
`adaptive_concave_threshold`	0.000	1.000	0.750	0.583 ± 0.425
`adaptive_tanh_episode`	**1.000**	**1.000**	**0.938**	$\mathbf{0.979 \pm 0.029}$

F3 — The robustness-ceiling tradeoff is fundamental. Wide support (large R): initialisation-robust, hover-prone. Narrow support (small R): hover-resistant, mode-switching unstable. No static geometry resolves both simultaneously. This tradeoff is not captured by potential-based shaping theory [2], which addresses asymptotic optimality but not training dynamics under finite samples.

F4 — Near-goal curvature does not independently predict performance. `tanh` (convex, $\nabla r|_{d=0} \neq 0$) and `tanh_sq` (concave, $\nabla r|_{d=0} = 0$) perform nearly identically across all seeds. H2 (stage-handoff geometry) is not confirmed as an independent effect; curvature matters only in conjunction with support structure.

F5 — Progress-gated support narrowing fails on hard seeds (chicken-and-egg). `adaptive_concave_threshold` ($R : 0.4 \rightarrow 0.1$, triggered when `success` ≥ 0.5) achieves 1.0 on seed 42 and 0.75 on seed 123, but 0.0 on seed 1: the wide initial support R=0.4 creates an even stronger hover attractor than R=0.2, so the threshold is never crossed and R never narrows.

F6 — Within-episode annealing resolves the tradeoff. `adaptive_tanh_episode` anneals the scale k from 2 to 20 over the max episode length T. Formally, at step $t \in [0, T]$:

$$r(d, t) = 1 - \tanh(k(t) \cdot d), \quad k(t) = 2 + 18 \cdot \frac{t}{T} \tag{5}$$

Early in each episode ($t \approx 0$), $k(t)$ is small, resulting in a wide gradient footprint that bootstraps exploration regardless of overall training progress. Late in each episode ($t \approx T$), $k(t)$ is large, concentrating the gradient strictly near the goal (the hover reward at $d = 0.2$m shrinks from $1 - \tanh(0.4) \approx 0.61$ to $1 - \tanh(4.0) \approx 0.004$). Because the curriculum is delivered within every episode, hard seeds get the wide bootstrap phase on every reset—breaking the chicken-and-egg. Results: mean 0.979 ± 0.029 versus next-best 0.646 ± 0.457 for `tanh_sq`.

6.2 Cross-Task Validation: PushCube-v1

PushCube-v1 requires the robot to push a cube to a target position without grasping—a flat task with no grasping stage. This tests whether the hover-attractor mechanism is specific to grasp-dependent tasks.

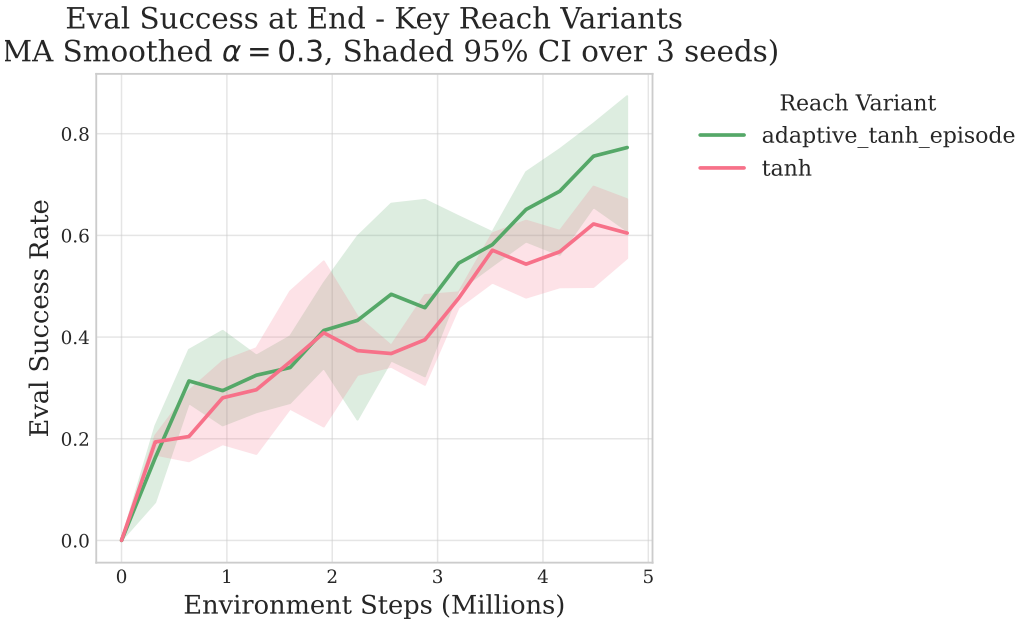

Figure 4: PushCube-v1: `adaptive_tanh_episode` maintains high performance (0.958 ± 0.029) compared to standard `tanh` (0.854 ± 0.128), demonstrating the robustness of within-episode annealing on a non-grasping task.

F7 — Hover attractor failure generalises to non-grasping tasks. `adaptive_tanh_episode` outperforms `tanh` on PushCube (0.958 vs. 0.854), confirming that the hover-attractor mechanism is not specific to the grasping stage but affects any manipulation task requiring precise end-effector positioning.

6.3 Cross-Task Validation: LiftPegUpright-v1

LiftPegUpright-v1 requires the robot to grasp a horizontal peg and reorient it to vertical—a task dominated by a *rotation* reward component (cosine similarity of peg orientation), with reaching contributing only $\frac{1}{5}$ of the shaping signal.

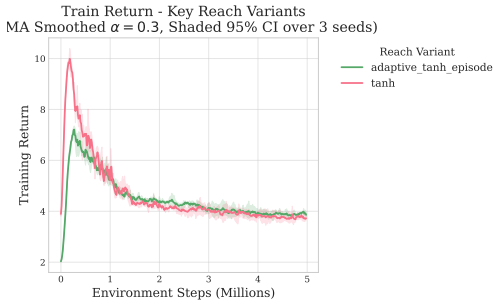

Figure 5: PushCube Training Returns

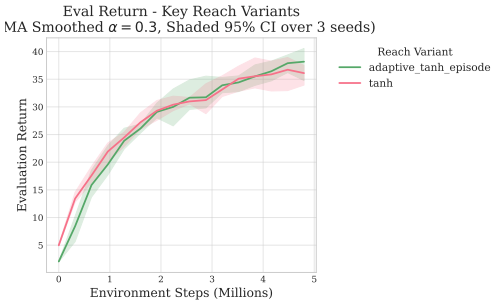

Figure 6: PushCube Eval Returns

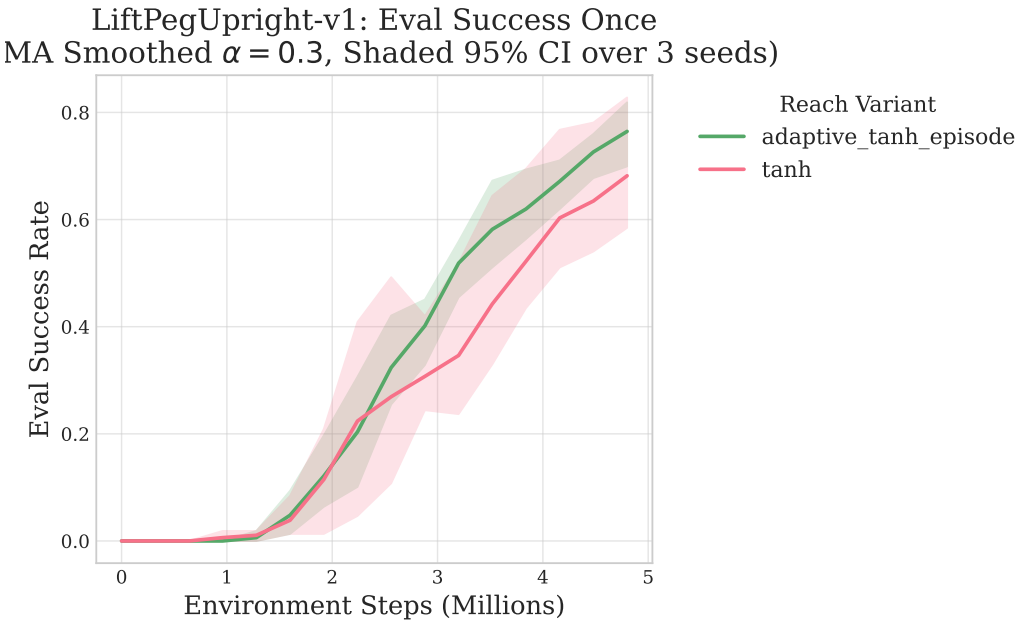

Figure 7: LiftPegUpright-v1: `eval/success_once` mean $\pm$ 95% CI over seeds $\{1, 42, 123\}$. Both variants reach similar final success rates, but `adaptive_tanh_episode` converges more reliably and shows lower inter-seed variance.

F8 — Hover attractor severity scales with reaching reward dominance. On PickCube, reaching is the primary shaping signal and the hover attractor completely blocks seed 1 (0.0 for `tanh`). On LiftPegUpright, the rotation reward dominates ($4\times$ weight of reaching), and seed 1 is no longer catastrophically stuck: both variants achieve 0.75 on seed 1. This confirms that the hover attractor is not a fixed property of the reaching function alone, but depends on its *relative weight* in the total reward signal. When rotation or placement rewards co-dominate, the reaching shaping function has less capacity to create a stable attractor.

F9 — Within-episode annealing provides consistent gains across task types. Despite the diluted reaching signal, `adaptive_tanh_episode` still outperforms `tanh` in mean success (0.854 vs 0.792) and reduces inter-seed variance. Notably, `eval/success_at_end`—a harder metric requiring the peg to remain upright at episode end—improves from 0.104 (`tanh`) to 0.167 (`adaptive_tanh_episode`), a $\approx 60\%$ relative gain, suggesting that within-episode annealing also helps commitment at task completion regardless of task structure.

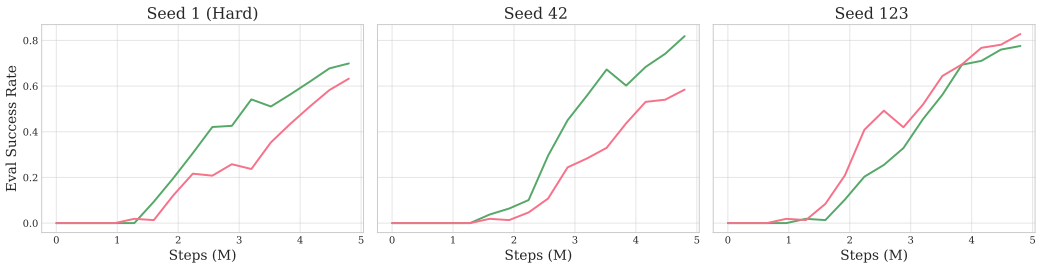

Figure 8: LiftPegUpright-v1 per-seed curves. Unlike PickCube, seed 1 is no longer a hard seed for either variant, suggesting that the hover attractor mechanism is diluted when reaching is not the dominant reward component.

Table 4: Final `eval/success_once` at 5M steps on LiftPegUpright-v1.

Variant	**Seed 1**	**Seed 42**	**Seed 123**	**Mean ± Std**
`tanh`	0.750	0.688	0.938	0.792 ± 0.107
`adaptive_tanh_episode`	**0.750**	**1.000**	**0.812**	$\mathbf{0.854 \pm 0.108}$

6.4 Cross-Task Validation: StackCube-v1

StackCube-v1 is a two-stage hierarchical task: the robot must grasp a cube, lift it, and place it precisely on top of a second cube. Crucially, success requires sustained, coordinated control across *two* contact stages with tight spatial tolerances. Runs use 40M steps.

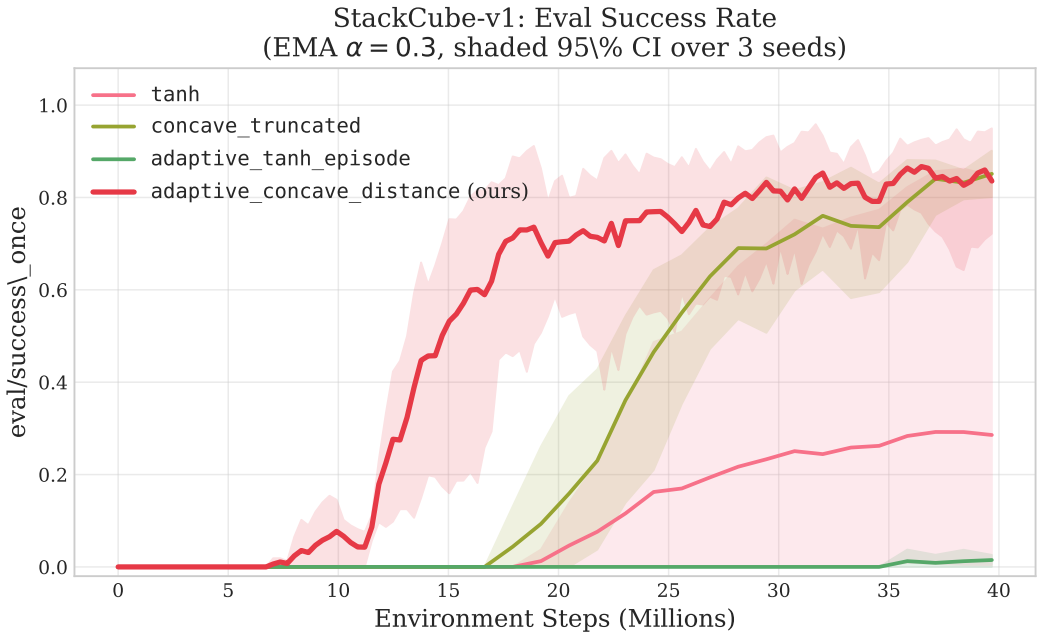

Figure 9: StackCube-v1: `eval/success_once` mean ± 95% CI over seeds $\{1, 42, 123\}$ (EMA smoothed α=0.3). `adaptive_concave_distance` and `concave_truncated` both reach high success; `adaptive_tanh_episode`—best on all flat tasks—collapses completely. `tanh` is seed-dependent (only seed 42 learns).

Figure 10: StackCube-v1 per-seed curves. `adaptive_concave_distance` reaches nonzero success on all three seeds by ∼7–12M steps, roughly 2× faster than `concave_truncated` (∼18–22M steps). `adaptive_tanh_episode` is at or near zero throughout on all seeds.

Table 5: StackCube-v1 at 40M steps: peak and final `eval/success_once`. Peak = max over training; Final = last checkpoint. Best mean peak in bold.

	Peak success			Final success		
Variant	**S1**	**S42**	**S123**	**S1**	**S42**	**S123**
`tanh`	0.063	0.938	0.063	0.000	0.812	0.000
`concave_truncated`	1.000	1.000	1.000	0.875	0.812	1.000
`adaptive_tanh_episode`	0.063	0.000	0.125	0.063	0.000	0.000
`adaptive_concave_distance`	**1.000**	**0.938**	**1.000**	1.000	0.750	0.938

Mean peak: `concave_trunc.` 1.000; `adaptive_concave_dist.` **0.979**; `tanh` 0.354; `adaptive_tanh_ep.` 0.063

F10 — Within-episode annealing catastrophically fails on hierarchical tasks. `adaptive_tanh_episode` achieves mean 0.021 on StackCube—virtually zero—despite being the best variant on all three flat tasks. This is a complete ranking inversion.

The mechanism is the interaction between within-episode annealing and hierarchical task structure. In PickCube, the task is effectively *flat*: the robot reaches, grasps, and places in one continuous motion, so wall-clock episode time ≈ task progress. In StackCube, the task has *two committed contact stages*: grasp the lower cube, then place it on the upper cube. The within-episode annealing schedules k as a function of *time*, not *task progress*. By the mid-episode point when k is already large, the robot may still be in stage 1 (reaching for the lower cube), and the now-tight reaching gradient provides insufficient signal for a robot even slightly off-center. The schedule mismatch prevents the policy from ever consolidating a reliable grasp, collapsing performance.

F11 — Finite support enables hierarchical manipulation; time-based annealing does not. `concave_truncated` achieves peak success 1.000 on all three seeds (Table 5), confirming that finite support is sufficient for hierarchical tasks. The zero-gradient zone at $d < R$ may actively *help*: once the robot is within reach, the reaching reward goes silent and stops competing with the downstream grasping and placement signal, allowing the policy to commit to contact. This is the inverse of the hover-attractor failure: reward *silence* near the goal creates a stable commitment region. However, `concave_truncated` is slow to learn: the first nonzero success does not appear until ≈ 18–22M steps across seeds.

F12 — Distance-adaptive reward matches peak performance with ≈2× faster convergence. `adaptive_concave_distance` achieves peak `eval/success_once` of 1.000 / 0.938 / 1.000 on seeds 1, 42, 123 (mean peak **0.979**), comparable to `concave_truncated` (mean peak 1.000), but with all three seeds showing nonzero success by ∼7–12M steps versus ∼18–22M steps for `concave_truncated` — roughly 2× faster onset. The final values (1.000 / 0.750 / 0.938) reflect late-training instability on seed 42, not a failure to learn: seed 42 peaks at 0.938 at ∼20M steps before partially decaying.

The mechanism follows directly from Section 5. Far from the goal ($d \approx 0.3$–0.5m), $k(d) \approx k_{\min} = 2$ and the outer support radius $R \approx 0.5$m provides gradient signal even from hard initialisations. Close to the goal ($d < 0.05$m), $k(d) \approx k_{\max} = 20$ tightens the basin, suppressing the soft hover attractor. Neither regime is hardcoded to a time or stage: when a stage transition occurs and d jumps to a large value, the reward automatically widens back to the exploration regime — without any clock reset.

6.5 Summary: Task-Structure-Dependent Playbook

Table 6: Cross-task summary: mean peak `eval/success_once` over seeds $\{1, 42, 123\}$. Best result per task in bold. '—' = not yet evaluated.

Task	**Type**	`tanh`	`concave_trunc.`	`adapt. tanh ep.`	`adapt. conc. dist.`
PickCube-v1	Flat	0.646	0.583	**0.979**	—
PushCube-v1	Flat	0.854	—	**0.958**	—
LiftPegUpright-v1	Flat	0.792	—	**0.854**	—
StackCube-v1	Hierarchical	0.354	1.000	0.063	**0.979**

Two patterns emerge. First, on flat tasks, within-episode annealing (`adaptive_tanh_episode`) consistently dominates: it resolves the robustness-ceiling tradeoff by delivering a wide-to-tight curriculum within every episode, ensuring hard seeds always receive the wide bootstrap phase. Second, on the hierarchical StackCube task, time-based annealing catastrophically fails (mean peak 0.063) because the episode clock decouples from task progress at stage boundaries. Finite-support `concave_truncated` succeeds (mean peak 1.000) via reward silence near the goal, but converges slowly ($\sim$18–22M steps to first success).

This discrete, task-type-dependent observation raises a deeper question: is there a single reward function that (a) resolves the hover attractor, (b) provides wide support from hard initialisations, and (c) does not depend on any time or stage clock? The theory in Section 5 and the results in F12 show that `adaptive_concave_distance` is a positive answer: by coupling the support radius directly to the current distance d, it eliminates both failure modes without any internal state. On StackCube it achieves mean peak 0.979 while converging $\approx 2\times$ faster than `concave_truncated`. Cross-task validation on flat tasks is the immediate next experiment (Section 7).

7 Planned Experiments

7.1 Cross-Task Validation of `adaptive_concave_distance`

The most immediate open question is whether `adaptive_concave_distance` maintains the strong performance of `adaptive_tanh_episode` on flat tasks. On a single-stage task, the distance d decreases monotonically throughout the episode, so the wide-to-tight transition of $k(d)$ mirrors the wide-to-tight annealing of `adaptive_tanh_episode` — but gated by actual proximity rather than time. We predict no regression on PickCube, PushCube, or LiftPegUpright.

Success criterion: `adaptive_concave_distance` achieves mean peak ≥ 0.93 on PickCube-v1 (matching `adaptive_tanh_episode`'s 0.979 within tolerance) without any catastrophic failure seeds, and ≥ 0.93 on PushCube-v1. If confirmed, it becomes a *universal* base reward: no task-type selection required.

7.2 Parameter Sensitivity ($k_{\min}$, $k_{\max}$, α)

The current configuration ($k_{\min} = 2$, $k_{\max} = 20$, $\alpha = 10$) is the first we tested. Section 5 derives the moat-existence condition $k_{\max}/k_{\min} > 1 + e^2 \approx 8.4$ and characterises the role of each parameter. Key ablations:

- Vary $k_{\max} \in \{14, 16, 18, 22\}$ to move across the moat threshold and assess whether the moat is beneficial, neutral, or harmful.

- Vary $k_{\min} \in \{1, 2, 3\}$ to change the outer support radius $R_{\text{outer}} = 1/k_{\min} \in \{1.0, 0.5, 0.33\}$m.
- Vary $\alpha \in \{5, 10, 15\}$ to shift the moat location and transition sharpness.

These ablations will reveal which aspects of the geometry drive performance and provide guidance for applying the reward to new tasks with different length scales.

7.3 Deeper Hierarchies: PlaceSphere-v1

PlaceSphere-v1 is a three-stage task (reach, grasp, place) with 13 reward components. It is a stronger test of distance-adaptive rewards on hierarchical tasks because stage transitions cause d to jump *twice*, each time returning $k(d)$ to the wide-support regime. We plan 3 seeds comparing `tanh`, `concave_truncated`, and `adaptive_concave_distance`.

7.4 Gating and Terminal Dimensions

Fix `adaptive_concave_distance` as the reaching variant and sweep the gating and terminal bonus dimensions on PickCube-v1. Cross-validate top configurations on StackCube-v1 and PlaceSphere-v1.

7.5 Algorithm Generalization

Repeat the top configurations with SAC and a model-based method to assess whether the geometric findings are specific to on-policy PPO or more general.

7.6 Sim-to-Real (Stretch)

Validate the top configuration on the SO-101 robotic arm via the `lerobot-sim2real` bridge supported in ManiSkill3.

8 Discussion

From task-type selection to a universal geometry. The initial playbook finding—flat tasks favour time-based annealing, hierarchical tasks favour finite support—is real but incomplete. It describes a consequence of a deeper principle: reward support should be wide when the agent is far from the goal and tight when close. Time is a good proxy for distance on flat tasks (where d decreases monotonically during a well-behaved episode) but a poor proxy on hierarchical tasks (where d resets at each stage). `adaptive_concave_distance` removes the proxy entirely by coupling the support radius directly to d, yielding a reward that is self-adapting, stateless, and task-structure-agnostic in principle.

The reward geometry perspective. Existing reward design literature focuses on magnitude (reward weights, normalisation) or potential-based structure (shaping theorems). The present work argues that *geometry*—the shape of the reward as a function of distance—is an independently important design axis. The hover attractor, the mode-switching instability, the schedule-mismatch failure, and the faster convergence of `adaptive_concave_distance` are all consequences of reward geometry rather than magnitude. Potential-based theory [2] is silent on all of them.

Limitations and open questions. The parameter triple $(k_{\min}, k_{\max}, \alpha)$ introduces new hyperparameters. We have only tested one configuration; Section 7 describes a systematic ablation. The moat condition $k_{\max}/k_{\min} > 1 + e^2$ is a precise analytical criterion but whether the moat is beneficial, neutral, or harmful in practice is an open empirical question. Finally, our experiments use PPO exclusively; off-policy methods may interact differently with reward geometry because they do not replay the same near-goal trajectories within a single update batch.

REP units as a reward design language for long-horizon tasks. The REPCon and REPTan units introduced in Section 5 suggest a broader design language: a long-horizon task with n stages can

be equipped with n REP units, one per stage, each with its own $(k_{\min}, k_{\max}, \alpha)$ triple encoding that stage's geometric requirements. $k_{\min} \approx 1/d_{\max}$ sets the outer support to match the maximum starting distance for that stage; $k_{\max} \approx 1/\epsilon$ encodes the required precision at success; and α controls how sharply the reward transitions between the two regimes. The current work uses a single shared REPCon unit across all stages — the degenerate case — and already achieves strong results, suggesting the geometry generalises well. Per-stage tuning is a natural next step, particularly for tasks with heterogeneous subtask geometries (e.g. free-space reaching followed by precision insertion).

A further extension is to *infer* REP unit parameters from video demonstrations rather than specifying them manually. A demonstration provides a distance profile $d(t)$ for each subtask: the maximum distance reveals $k_{\min}$, the success tolerance reveals $k_{\max}$, and the sharpness of the $d(t)$ drop reveals α. Stage boundaries are identified as points where $d(t)$ jumps as the next subgoal activates. This would make the entire dense reward function inferrable from a small number of demonstrations with no reward engineering — connecting the REP unit framework to imitation learning and inverse reward design.

REP units on learned task-progress signals. Throughout this work d denotes Euclidean end-effector distance, which is a good proxy for inverse task progress on reaching tasks (where $d \to 0$ corresponds to task completion) but a poor one for tasks with strong orientation or contact requirements, such as peg insertion. A natural generalisation is to treat d as *any* scalar measure of inverse task progress — with $d = 0$ meaning the subtask is complete and $d \to \infty$ meaning it has barely begun. Under this interpretation, the REPCon geometry is immediately applicable to learned progress signals such as those produced by Robometer or TOPReward, which output a dense per-frame progress scalar from a small demonstration set. Substituting a learned progress signal into the REPCon formula would allow the geometric guarantees (outer exploration support, inner precision basin, no hover attractor) to be transferred to tasks where Euclidean distance fundamentally fails as a progress proxy, such as `PegInsertionSide-v1`. The parameter triple $(k_{\min}, k_{\max}, \alpha)$ retains its interpretation: $1/k_{\min}$ is the maximum starting progress distance, $1/k_{\max}$ is the success tolerance in progress units, and $1.65/\alpha$ is the commitment threshold — the progress level past which the reward switches from exploratory to precise.

Train/eval success gap as a reward quality diagnostic. Explained variance is uniformly ≈ 0.96 across all variants—uninformative as a discriminator. The train/eval success gap is more revealing: a large gap indicates the policy is exploiting the dense reward without achieving task completion under the deterministic evaluation policy. Figure 11 tracks this for the PickCube variants.

9 Conclusion

We have presented the Reward Engineering Playbook: a systematic study of how the geometry of dense reward functions—not their magnitude or potential structure—causally affects manipulation learning. Our key findings are:

1. **Hover attractors** are a systematic failure mode of infinite-support rewards, causing initialisation-dependent policy collapse on PickCube and PushCube.
2. **The robustness-ceiling tradeoff** between finite- and infinite-support geometry is fundamental: no static support radius resolves both the blind-region failure and the hover attractor simultaneously.
3. **Within-episode annealing** (`adaptive_tanh_episode`) resolves the tradeoff on flat tasks (mean peak 0.979 on PickCube) but causes a complete ranking inversion on hierarchical ones (mean peak 0.063 on StackCube), because the episode time clock decouples from task progress at stage boundaries.
4. `adaptive_concave_distance` — derived from the principle that support radius should track the agent's current distance — achieves mean peak 0.979 on StackCube with $\approx 2\times$ faster convergence than the best task-specific static reward, with no time clock or stage information required.

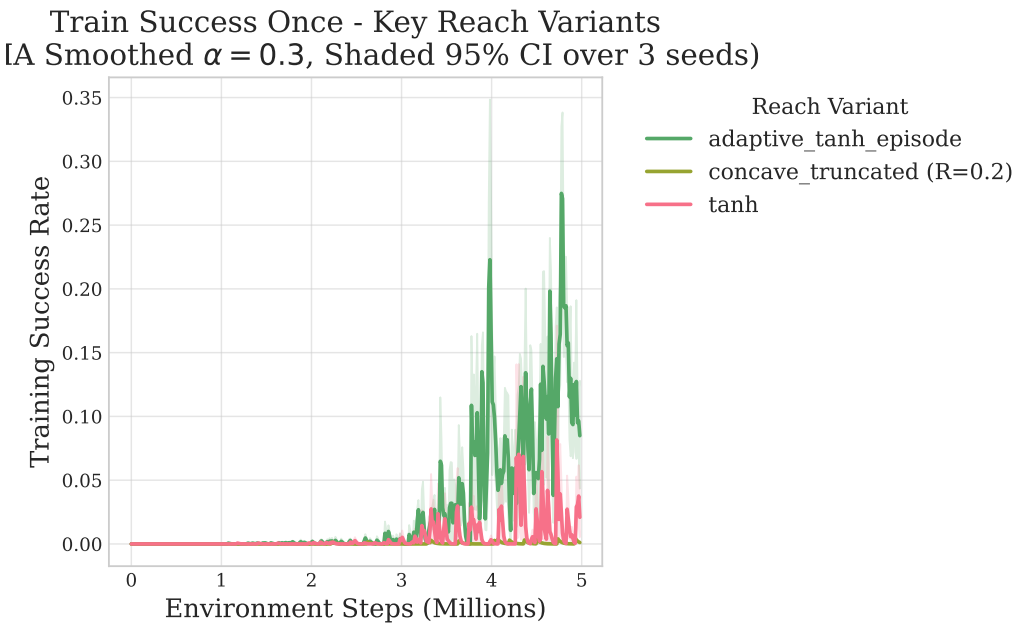

Figure 11: PickCube training success rates. Compare against Figure 1 (eval success) to observe the train/eval gap.

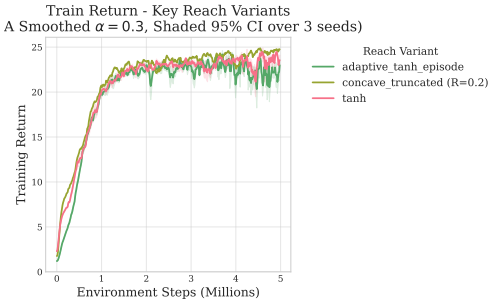

Figure 12: PickCube Training Returns

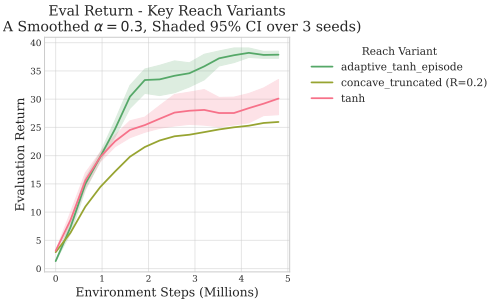

Figure 13: PickCube Evaluation Returns

The theoretical framework in Section 5 provides closed-form conditions characterising the reward's geometry as a function of $(k_{\min}, k_{\max}, \alpha)$, connecting the empirical results to an analytical foundation. Cross-task validation of `adaptive_concave_distance` on flat tasks and a parameter sensitivity study are the immediate next steps toward a fully unified, theoretically principled reward function for robot manipulation.

References

[1] Y. J. Ma, W. Liang, G. Shi, L. Fan, and A. Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

[2] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

[3] S. Devlin and D. Kudenko. Dynamic potential-based reward shaping. In *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, pages 433–440, 2012.

[4] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Origo, P. Humphreys, O. Bachem, and M. Geist. What matters in on-policy reinforcement learning? a large-scale empirical study. In *International Conference on Machine Learning (ICML)*, pages 520–531. PMLR, 2021.

[5] J. Gu et al. Maniskill: A unified gpu-parallelized environment for robotics manipulation. *arXiv preprint*, 2024.

[6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[7] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, pages 1861–1870. PMLR, 2018.

A Supported Tasks and Stage Structure

Table 7 lists all 11 ManiSkill3 tabletop tasks instrumented with reward variants in this study, along with their episode length and stage structure. "Stage" refers to whether the task has a single committed reaching-and-contact phase (flat) or multiple committed contact stages with discrete transitions (hierarchical). Results are currently reported for PickCube, PushCube, LiftPegUpright, and StackCube; the remaining tasks are planned.

Table 7: ManiSkill3 tasks with reward variant support.

Task	**Stage structure**	**Max steps**	**Training steps**
PickCube-v1	Single (reach, grasp, lift)	50	5M
PushCube-v1	Single (reach push, push to goal)	50	5M
LiftPegUpright-v1	Single (reach, grasp, orient)	50	5M
PullCube-v1	Single (reach, pull to goal)	50	5M
PokeCube-v1	Single (reach, poke to position)	50	5M
RollBall-v1	Single (reach, roll to goal)	80	5M
PickSingleYCB-v1	Single (reach, grasp, lift)	50	5M
PlaceSphere-v1	Hierarchical (reach, grasp, place)	50	5M
PullCubeTool-v1	Hierarchical (grasp tool, pull cube)	100	5M
StackCube-v1	Hierarchical (grasp, stack)	50	40M
PegInsertionSide-v1	Hierarchical (reach, grasp, orient, insert)	100	**[TODO: TBD]**